Lab 1 — Lecture Slides (Deck A)

Course: CS-2007L Data Structures Lab (C#)
Audience: Students (project this deck)
Language: English
Style: One idea per slide. Say the Roman Urdu script aloud; keep slides short.

Slide 1 — Title

CS-2007L · Lab 1
Introduction to the C# Environment & Basic Data Organization

Week 1 · 3 hours

Slide 2 — Today's outcomes

By the end of this lab you will be able to:

1. Create, build, run, and debug a .NET 8 console project
2. Explain data → field → record → file
3. Contrast value types vs reference types
4. Perform six array operations
5. Measure a simple time–space tradeoff
6. Document code + disclose AI use honestly

Slide 3 — Why this lab?

Real setting	Data structure idea
Course enrolment list	insert, delete, search, sort
Sensor ID stream (factory)	traverse, find duplicates

This semester: arrays → lists → stacks/queues → trees → graphs → hashing.

Slide 4 — From source to running program

```
Program.cs  →  compiler  →  Lab01.dll (IL)  →  CLR + JIT  →  machine code  →  output
```

- **Compiler:** C# → Intermediate Language
- **CLR:** runs IL, JIT, garbage collection

Slide 5 — Tools you need

- **.NET 8.0 SDK** (LTS)
- **IDE:** Visual Studio 2022 *or* VS Code + C# Dev Kit
- Terminal: Command Prompt / PowerShell / IDE terminal

Check: `dotnet --version` → should start with `8.0`

Slide 6 — Essential commands

Command	Purpose
`dotnet --version`	Show SDK
`dotnet new console -n Lab01 --use-program-main`	New project with `Main`
`dotnet build`	Compile
`dotnet run`	Build (if needed) + run

Never submit `bin/` or `obj/`.

Slide 7 — Project anatomy

- `Program.cs` — your code
- `Lab01.csproj` — framework & settings
- `bin/`, `obj/` — generated (ignore for submission)

.NET 8 implicit usings: `System`, `System.IO`, `System.Linq`, `System.Collections.Generic` are often already imported.

Slide 8 — LIVE: Hello, C# (Example 1)

Type it yourself (do not paste).

- `namespace` groups types
- `Main` is the entry point
- `$"..."` string interpolation
- `///` XML documentation comments

Expected: Hello lines + `.NET 8.0.x`

Slide 9 — Three kinds of errors

Kind	When	Example
Syntax / compile	Build fails	Typo → CS0103
Runtime	Throws while running	`IndexOutOfRangeException`
Logic	Runs, wrong answer	Off-by-one in a loop

Debugger is for **logic** errors.

Slide 10 — Debugger keys

Action	VS / VS Code
Start debugging	F5

Toggle breakpoint	F9
Step over / into	F10 / F11
Inspect	Locals / Watch

Slide 11 — LIVE: SumToN bug (Example 2)

Expected sum `1..5` = **15**
Program prints **10**.

1. Breakpoint on `sum += i`
2. Watch `i` and `sum`
3. Count loop iterations
4. Fix: `i <= n`

Compiler was happy. Testing + debugger found the bug.

Slide 12 — Data organization terms

Term	Meaning	Example
Data	Raw fact	`"Ayesha"`, `3.45`
Field	Named attribute	`Name`, `Cgpa`
Record	Related fields (one entity)	one student
File	Related records on disk	`students.txt`

Slide 13 — Value vs reference

Value type (`int` , `double` , ...): assignment copies the **value**.

Reference type (arrays, classes): assignment copies the **reference** (same object on the heap).

`Clone()` → independent array copy.
`string` is reference but **immutable**.

Slide 14 — LIVE: TypesDemo (Example 3)

Watch for:

- `a` stays 10 when `b` changes
- `first[0]` changes when `second[0]` changes
- `Clone()` keeps `first` independent
- `int.MaxValue + 1` wraps (unchecked)

Slide 15 — LIVE: Record + file (Example 4)

`StudentRecord` = fields → one record
Array of records in memory → write CSV lines → `students.txt` = file

Open the file in Notepad and confirm.

Slide 16 — Six operations on an array

1. Traverse
2. Search
3. Insert
4. Delete
5. Sort
6. Merge

Slide 17 — Typical costs (array)

Op	Typical cost
Access by index	O(1)
Traverse / linear search	O(n)
Insert / delete	O(n) (shift / copy)
Good sort	O(n log n)
Merge two sorted	O(n + m)

Lab 2: Big-O formally.

Slide 18 — LIVE: ArrayOperations (Example 5)

Follow the demo output.

Try `Insert(data, 10, 1)` → exception → wrap with `try/catch` .

Note: Merge assumes **both inputs are already sorted**.

Slide 19 — Time–space tradeoff

Often: **more memory** → **faster** (or less memory → slower).

Example: many membership queries

- Linear scan: little extra space, slow
- `HashSet` : extra space, near O(1) lookup

Measure with `Stopwatch` in **Release**: `dotnet run -c Release`

Slide 20 — LIVE: TradeoffDemo (Example 6)

Same answers from both methods.
Compare milliseconds.
Discuss: cloud server vs memory-limited edge device.

Slide 21 — Graded tasks today

In lab (priority): Tasks 1–4

#	Task	Focus
1	Environment report	Toolchain
2	Student profile card	Input + `TryParse`
3	Types table + classification	CLO-1 written
4	BuggyStats	Debugger screenshots

If time / homework: Tasks 5–6 + post-lab Q1–Q3

Slide 22 — Submit & next week

- Code builds; screenshots of code + output
- Task 4: one debugger screenshot per bug
- PDF to LMS; name + reg no on top
- Honest **AI-use disclosure**
- Do not submit `bin` / `obj`

Next: Lab 2 — frequency count, Big-O, measured growth

Presenter checklist

- [ ] SDK verified on lab PCs / students' machines
- [ ] Example 2 breakpoint demo rehearsed
- [ ] Release build note for Example 6
- [ ] Rubric ready (10 marks: 2+3+2+3)